

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Избранные главы информатики»

ОТЧЕТ

к лабораторной работе №4

на тему:

**«Работа с файлами, классами, сериализаторами, регулярными
выражениями и стандартными библиотеками»**

БГУИР КП 6-05 0612 02 060 ПЗ

Выполнила студентка группы 453502
КРУЧЁНОК Александра Сергеевна

Проверил канд. т. н., доцент
ЖВАКИНА Анна Васильевна

(дата, подпись преподавателя)

Минск 2026

Цель: освоить базовый синтаксис языка Python, приобрести навыки работы с файлами, классами, сериализаторами, регулярными выражениями и стандартными библиотеками и закрепить их на примере разработки интерактивных приложений.

Вариант 13

Ссылки:

1. <https://docs.python.org/> - офф документация (англ)
2. <https://pythonworld.ru/> - для начинающих (рус)
3. <https://smartiga.ru/courses/python/lesson-1> - смартика (годно)
4. <https://pythonru.com/uroki/vvedenie-uroki-po-python-dlja-nachinajushhih> - краткий питон (рус)
5. <https://pythontutor.ru/> - тренажер
6. <https://pythonchik.ru/osnovy> - учебник (рус)
7. <https://younglinux.info/python/course> - онлайн-учебник (рус)
8. <https://www.w3schools.com/python/default.asp> - тесты + теория (англ)

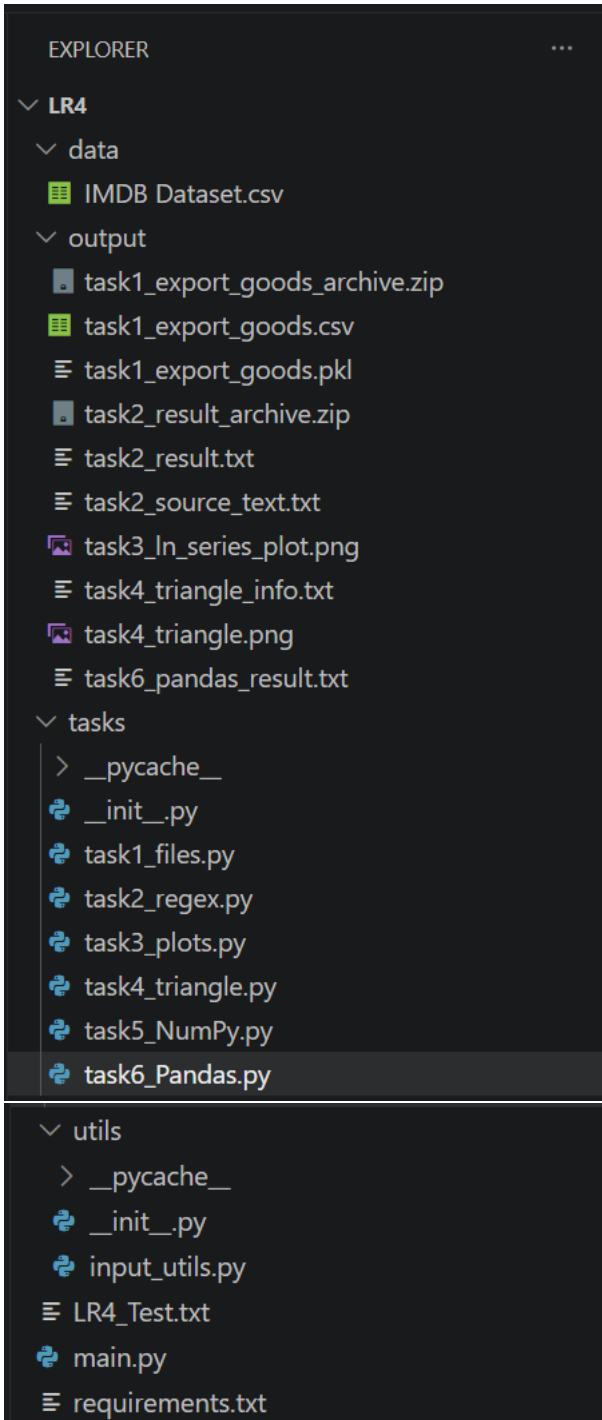
Для защиты ЛР необходимо оформить отчет со скринами кода и результатов его выполнения.

Требования к выполнению

1. Программа должна быть снабжена комментариями на английском языке, в которых необходимо указать краткое предназначение программы, номер лабораторной работы и название, версию программы, Ф.И.О. разработчика и дату разработки. Выполнить документирование кода для получения справки по каждой функции
2. Программа должна быть снабжена дружелюбным и интуитивно понятным интерфейсом
3. Для выполнения каждого задания разработать пользовательские классы и обеспечить их наследование. Реализовать примеры использования:
 - статических и динамических атрибутов класса,
 - полиморфизма,
 - специальных(магических методов),
 - `super()`,
 - геттеров и сеттеров,
 - свойств класса,
 - примесей.
4. При разработке программ рекомендуется придерживаться принципа: за решение одной конкретной задачи должна отвечать одна функция.
5. Все функции необходимо сгруппировать в модулях, согласно их логике их работы.
6. Разработанные основные функции, размещенные в отдельных модулях, нужно подключить в другом модуле, где будет происходить тестирование данных функций.
7. В программах предусмотреть возможность повторного выполнения без выхода из программы и защиту от ввода некорректных пользовательских данных. Для этих целей рекомендуется разработать отдельные функции. Обеспечить обработку конкретных классов исключений.

Индивидуальные задания по вариантам

Структура всего проекта:



Модуль тестирования всей программы – **main.py**

Обработка ввода и возможных ошибок – **input_utils.py**

!!! В каждом листинге программ присутствуют комментарии по основным поинтам заданий для удобного процесса сдачи.

Листинг main.py:

```
"""Main module for Laboratory Work No. 4.

Laboratory work title:
"Working with files, classes, serializers, regular expressions and standard libraries"

Version: 1.6
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-04-22
"""

from tasks.task1_files import run_task_1
from tasks.task2_regex import run_task_2
from tasks.task3_plots import run_task_3
from tasks.task4_triangle import run_task_4
from tasks.task5_NumPy import run_task_5
from tasks.task6_Pandas import run_task_6
from utils.input_utils import ask_int_in_range


def show_menu() -> None:
    """Display the main application menu."""
    print("\n" + "=" * 80)
    print("Laboratory work No. 4")
    print("Theme: Working with files, classes, serializers, regular expressions")
    print("Developer: Kruchonok Aleksandra Sergeevna")
    print("=" * 80)
    print("1. Task 1 - CSV, pickle, zipfile, exported goods")
    print("2. Task 2 - Regular expressions and text analysis")
    print("3. Task 3 - ln(1 + x), class, statistics and matplotlib")
    print("4. Task 4 - Triangle class hierarchy and drawing")
    print("5. Task 5 - NumPy matrix processing")
    print("6. Task 6 - Pandas IMDB dataset analysis")
    print("0. Exit")
    print("=" * 80)


def main() -> None:
    """Start the menu-driven application."""
    actions = {
        1: run_task_1,
        2: run_task_2,
        3: run_task_3,
        4: run_task_4,
        5: run_task_5,
        6: run_task_6,
    }
```

```
while True:
    show_menu()

    choice = ask_int_in_range("Choose task number: ", 0, 6)

    if choice == 0:
        print("Program finished. Goodbye!")
        break

    try:
        actions[choice]()
    except KeyboardInterrupt:
        print("\nTask interrupted by user. Returning to main menu.")
    except Exception as error:
        print(f"Unexpected error: {error}")

if __name__ == "__main__":
    main()
```

Листинг input_utils.py:

```
"""Input utility functions for Lab 4.

Lab title:
"Working with files, classes, serializers, regular expressions and standard libraries"

Version: 1.1
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-04-10
"""

from __future__ import annotations

def ask_yes_no(message: str) -> bool:
    """Ask the user a yes/no question.

    Args:
        message: Text displayed to the user.

    Returns:
        True if the user enters 'y' or 'yes', otherwise False.
    """
    while True:
        value = input(message).strip().lower()

        if value in {"Y", "y", "yes", "д", "да"}:
            return True

        if value in {"N", "n", "no", "н", "нет"}:
            return False

        print("Input error: enter y/n.")

def ask_int(message: str) -> int:
    """Read an integer value from the keyboard.

    Args:
        message: Text displayed to the user.

    Returns:
        Integer value entered by the user.
    """
    while True:
        try:
            return int(input(message).strip())
        except ValueError:
```

```

        print("Input error: enter an integer number.")

def ask_int_in_range(message: str, min_value: int, max_value: int) -> int:
    """Read an integer value from the selected range.

    Args:
        message: Text displayed to the user.
        min_value: Minimum allowed value.
        max_value: Maximum allowed value.

    Returns:
        Correct integer value.
    """
    while True:
        value = ask_int(message)

        if min_value <= value <= max_value:
            return value

        print(f"Input error: enter a number from {min_value} to {max_value}.")

def ask_float(message: str) -> float:
    """Read a float value from the keyboard.

    Args:
        message: Text displayed to the user.

    Returns:
        Float value entered by the user.
    """
    while True:
        try:
            return float(input(message).strip().replace(",", "."))
        except ValueError:
            print("Input error: enter a real number.")

def ask_positive_float(message: str) -> float:
    """Read a positive float value from the keyboard.

    Args:
        message: Text displayed to the user.

    Returns:
        Positive float value.
    """
    while True:

```

```
    value = ask_float(message)

    if value > 0:
        return value

    print("Input error: value must be positive.")

def ask_non_empty_string(message: str) -> str:
    """Read a non-empty string from the keyboard.

    Args:
        message: Text displayed to the user.

    Returns:
        Non-empty string.
    """
    while True:
        value = input(message).strip()

        if value:
            return value

    print("Input error: value cannot be empty.")
```


Задание 1. Исходные данные представляют собой словарь. Необходимо поместить их в файл, используя сериализатор. Организовать считывание данных, поиск, сортировку в соответствии с индивидуальным заданием. Обязательно использовать классы. Реализуйте два варианта:

- 1) формат файлов CSV;
- 2) модуль pickle.

Вар-т	Условие
13	В сводке об экспортируемых товарах указывается: наименование товара, страна , импортирующая товар, объем поставляемой партии в штуках . Напечатайте <u>списки стран</u> , в которые экспортируется данный товар, и <u>общий объем его экспорта</u> . Выведите <u>информацию о товаре, введенном с клавиатуры</u>

Листинг основного кода программы (+ добавлена работа с архивом):

```
"""Laboratory Work No. 4.

Task 1. CSV and pickle serialization.

Variant 13:
Exported goods summary:
product name, importing country, shipment amount in pieces.

Version: 1.2
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-04-11
"""

# Исходные данные - словарь. Поместить их в файл (сериализаторы pickle и CSV).
# Организовать считывание данных, поиск, сортировку.
# Сводка об экспортируемых товарах:
# наименование товара, страна, импортирующая товар, объем партии в штуках.
# Напечатать списки стран и общий объем его экспорта.
# Вывести инфу о товаре, введенном с клавиатуры

from __future__ import annotations

import csv
import pickle
import zipfile
from dataclasses import dataclass
from pathlib import Path

from utils.input_utils import ask_non_empty_string, ask_yes_no

OUTPUT_DIR = Path("output")

CSV_FILE = OUTPUT_DIR / "task1_export_goods.csv"
```

```

PICKLE_FILE = OUTPUT_DIR / "task1_export_goods.pkl"
ARCHIVE_FILE = OUTPUT_DIR / "task1_export_goods_archive.zip"

CSV_FIELDS = ["product", "country", "amount"]

# Если архивирование не нужно, False.
CREATE_ARCHIVE = True

@dataclass
class ExportRecord:
    """Store one export record."""

    product: str
    country: str
    amount: int

    # метод экземпляра
    def to_dict(self) -> dict[str, str | int]:
        """Convert the object to a dictionary."""
        return {
            "product": self.product,
            "country": self.country,
            "amount": self.amount,
        }

    # метод класса
    @classmethod
    def from_dict(cls, row: dict[str, str]) -> "ExportRecord":
        """Create an object from a dictionary."""
        return cls(
            product=row["product"],
            country=row["country"],
            amount=int(row["amount"]), # из CSV все значения читаются как строки
        )

class ExportCatalog:
    """Store records and perform search, sorting and calculations."""

    # инициализатор объекта (заполняет объект данными после создания)
    def __init__(self, records: list[ExportRecord]) -> None:
        """Initialize the catalog."""
        self.records = records

    # Поиск (сравнивает введенный товар с названием каждого товара)
    def find(self, product_name: str) -> list[ExportRecord]:
        """Find records by product name."""
        product_name = product_name.strip().lower() # lower - не зависеть от регистра

```

```

        return [
            record
            for record in self.records
            if record.product.lower() == product_name
        ]

# Список стран и общий объем
def countries_and_total(self, product_name: str) -> tuple[list[str], int]:
    """Return importing countries and total export amount."""
    found_records = self.find(product_name)
    countries = sorted({record.country for record in found_records})
    total = sum(record.amount for record in found_records)

    return countries, total

# сортировка по названию товара, потом по стране, по объему
def sorted_records(self) -> list[ExportRecord]:
    """Return records sorted by product, country and amount."""
    return sorted(
        self.records,
        key=lambda record: (
            record.product.lower(),
            record.country.lower(),
            record.amount,
        ),
    )

def create_catalog() -> ExportCatalog:
    """Create source data."""
    source_data = {
        "wheat": [("Poland", 1200), ("Germany", 850), ("Italy", 730)],
        "potatoes": [("Lithuania", 600), ("Latvia", 410), ("Poland", 900)],
        "tractors": [("Kazakhstan", 35), ("Armenia", 17), ("Georgia", 21)],
        "linen": [("France", 320), ("Germany", 280), ("Spain", 175)],
        "cheese": [("Poland", 500), ("Lithuania", 240), ("Czech Republic", 310)],
    }
    records = []

    for product, deliveries in source_data.items():
        for country, amount in deliveries:
            records.append(ExportRecord(product, country, amount))

    return ExportCatalog(records)

def save_to_csv(catalog: ExportCatalog) -> None:
    """Save records to a CSV file."""
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

```

```

# w автоматически перезаписывает файл, если он уже существует
with CSV_FILE.open("w", encoding="utf-8", newline="") as file:
    # запись словаря в CSV (1 метод CSV)
    writer = csv.DictWriter(file, fieldnames=CSV_FIELDS)
    # запись в CSV первой строки - заголовков столбцов таблицы (3 метод CSV)
    writer.writeheader()

    for record in catalog.records:
        # запись в CSV словаря построчно (4 метод CSV)
        writer.writerow(record.to_dict())

# СЧИТЫВАНИЕ
def load_from_csv() -> ExportCatalog:
    """Load records from a CSV file."""
    records = []

    with CSV_FILE.open("r", encoding="utf-8", newline="") as file:
        # читает CSV-файл построчно и представляет каждую строку как словарь (2 метод CSV)
        reader = csv.DictReader(file)

        for row in reader:
            records.append(ExportRecord.from_dict(row))

    return ExportCatalog(records)

def save_to_pickle(catalog: ExportCatalog) -> None:
    """Save records using pickle serializer."""
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    #Сериализация (pickle dump)
    # Режим "wb" автоматически перезаписывает бинарный файл.
    with PICKLE_FILE.open("wb") as file:
        pickle.dump(catalog.records, file)

# СЧИТЫВАНИЕ
def load_from_pickle() -> ExportCatalog:
    """Load records from a pickle file."""

    #Десериализация (pickle load)
    with PICKLE_FILE.open("rb") as file:
        records = pickle.load(file)

    return ExportCatalog(records)

# архивирование
def create_archive() -> None:

```

```

"""Create ZIP archive and print compression percent."""
# mode="w" автоматически перезаписывает архив, если он уже существует.
with zipfile.ZipFile(
    ARCHIVE_FILE,
    mode="w", # запись в архив
    compression=zipfile.ZIP_DEFLATED, # алгоритм сжатия
    compresslevel=9, # макс степень сжатия
) as archive:
    archive.write(CSV_FILE, arcname=CSV_FILE.name)
    archive.write(PICKLE_FILE, arcname=PICKLE_FILE.name)

print("\nArchive information:")

# чтение из архива
with zipfile.ZipFile(ARCHIVE_FILE, mode="r") as archive:
    for file_info in archive.infolist():
        original_size = file_info.file_size
        compressed_size = file_info.compress_size

        if original_size == 0:
            compression_percent = 0
        else:
            # расчет % сжатия
            compression_percent = (1 - compressed_size / original_size) * 100

        print("-" * 60)
        print(f"File in archive: {file_info.filename}")
        print(f"Original size: {original_size} bytes")
        print(f"Compressed size: {compressed_size} bytes")
        print(f"Compression percent: {compression_percent:.2f}%")

# получаем инфу о файлах без распаковки
csv_info = archive.getinfo(CSV_FILE.name)
print("-" * 60)
print("Information received by getinfo() without extracting the file:")
print(f"{csv_info.filename}: {csv_info.file_size} bytes")

def print_records(records: list[ExportRecord]) -> None:
    """Print records as a table."""
    print("\n" + "-" * 65)
    print(f"{'Product':<18} {'Country':<25} {'Amount':>10}")
    print("-" * 65)

    for record in records:
        print(f"{record.product:<18} {record.country:<25} {record.amount:>10}")

    print("-" * 65)

```

```

def process_search(catalog: ExportCatalog) -> None:
    """Search product information entered by the user."""
    product_name = ask_non_empty_string("\nEnter product name: ")
    found_records = catalog.find(product_name)

    if not found_records:
        print("No records found for this product.")
        return

    countries, total = catalog.countries_and_total(product_name)
    print_records(found_records)
    print(f"Countries importing '{product_name}': {'', ' '.join(countries)}")
    print(f"Total export amount: {total} pieces")

def run_task_1() -> None:
    """Run task 1."""
    while True:
        print("\nTask 1. Exported goods, CSV and pickle.")
        print("Developer: Kruchonok Aleksandra Sergeevna")
        print("Development date: 2026-04-11")

        catalog = create_catalog()

        save_to_csv(catalog)
        save_to_pickle(catalog)

        # CSV считывается корректно.
        csv_catalog = load_from_csv()

        # Pickle тоже считывается, но второй раз таблицу не выводим.
        pickle_catalog = load_from_pickle()

        print("\nData loaded from CSV and sorted (Pickle - same, test in code):")
        print_records(csv_catalog.sorted_records())

        print(f"\nRecords loaded from pickle: {len(pickle_catalog.records)}")

        process_search(csv_catalog)

        if CREATE_ARCHIVE:
            create_archive()
            print(f"\nArchive file: {ARCHIVE_FILE}")

        print(f"\nCSV file: {CSV_FILE}")
        print(f"Pickle file: {PICKLE_FILE}")
        if not ask_yes_no("\nRepeat task 1? (y/n): "):
            break

```

Работа программы (демонстрационный ввод – cheese):

```
PS C:\Users\Sanechka\Desktop\453502_KRUCHONOK_13\IGI\LR4> & C:\Users\Sanechka\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Sanechka/Desktop/453502_KRUCHONOK_13/IGI/LR4/main.py
```

```
=====
Laboratory work No. 4
Theme: Working with files, classes, serializers, regular expressions
Developer: Kruchonok Aleksandra Sergeevna
=====
```

1. Task 1 - CSV, pickle, zipfile, exported goods
2. Task 2 - Regular expressions and text analysis
3. Task 3 - $\ln(1 + x)$, class, statistics and matplotlib
4. Task 4 - Triangle class hierarchy and drawing
5. Task 5 - NumPy matrix processing
6. Task 6 - Pandas IMDB dataset analysis
0. Exit

```
=====
Choose task number: 1
```

```
Task 1. Exported goods, CSV and pickle.
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-04-11
```

```
Data loaded from CSV and sorted (Pickle - same, test in code):
```

Product	Country	Amount
cheese	Czech Republic	310
cheese	Lithuania	240
cheese	Poland	500
linen	France	320
linen	Germany	280
linen	Spain	175
potatoes	Latvia	410
potatoes	Lithuania	600
potatoes	Poland	900
tractors	Armenia	17
tractors	Georgia	21
tractors	Kazakhstan	35
wheat	Germany	850
wheat	Italy	730
wheat	Poland	1200

```
Records loaded from pickle: 15
```

```
Enter product name: cheese
```

Product	Country	Amount
cheese	Poland	500
cheese	Lithuania	240
cheese	Czech Republic	310

```
-----
Countries importing 'cheese': Czech Republic, Lithuania, Poland
Total export amount: 1050 pieces
```

```

Archive information:
-----
File in archive: task1_export_goods.csv
Original size: 333 bytes
Compressed size: 192 bytes
Compression percent: 42.34%
-----
File in archive: task1_export_goods.pkl
Original size: 544 bytes
Compressed size: 314 bytes
Compression percent: 42.28%
-----
Information received by getinfo() without extracting the file:
task1_export_goods.csv: 333 bytes

Archive file: output\task1_export_goods_archive.zip

CSV file: output\task1_export_goods.csv
Pickle file: output\task1_export_goods.pkl

Repeat task 1? (y/n): n
=====

```

Демонстрация dumps и loads (не вошло в листинг кода):

```

def demonstrate_pickle_dumps_loads(database: ExportDatabase) -> None:
    """Demonstrate pickle.dumps and pickle.loads.
    Args:
        database: ExportDatabase object.
    """
    # dumps не работает с файлом.
    # dumps превращает объект в bytes внутри программы.
    binary_data = pickle.dumps(database.records)

    # loads восстанавливает объект из bytes внутри программы.
    restored_records = pickle.loads(binary_data)

    print("\nPickle dumps/loads demonstration:")
    print(f"Bytes object size after dumps: {len(binary_data)} bytes")
    print(f"Restored records after loads: {len(restored_records)}")

```


Задание 2. В соответствии с заданием своего варианта составить программу для анализа текста. Считать из исходного файла текст. Используя регулярные выражения получить искомую информацию (см. условие), вывести ее на экран и сохранить в другой файл. Заархивировать файл с результатом с помощью модуля zipfile и обеспечить получение информации о файле в архиве.

Также выполнить общее задание – определить и сохранить в файл с результатами:

- количество предложений в тексте;
- количество предложений в тексте каждого вида отдельно (повествовательные, вопросительные и побудительные);
- среднюю длину предложения в символах (считаются только слова);
- среднюю длину слова в тексте в символах;
- количество смайликов в заданном тексте.

Смайликом будем считать последовательность символов, удовлетворяющую условиям:

- первым символом является либо «;» (точка с запятой) либо «:» (двоеточие) ровно **один раз**;
- далее может идти символ «-» (минус) **сколько угодно** раз (в том числе символ минус может идти **ноль** раз);
- в конце обязательно идет некоторое количество (**не меньше одной**) **одинаковых скобок** из следующего набора: «(», «)», «[», «]»; внутри смайлика не может встречаться никаких других символов.

Например, эта последовательность является смайликом: «;-----[[[[[[[».

Эти последовательности смайликами не являются: «]», «;--», «:», «)».

Вар-т	Условие
13	Получить список шестнадцатеричных чисел, содержащихся в тексте. Проверить, существуют ли в тексте цифры, за которыми стоит «+». Пример правильного выражения: $(3 + 5) - 9 \times 4$. Пример неправильного выражения: $2 * 9 - 6 \times 5$. Определить число слов, длина которых равна 4 символа; найти слова, у которых количество гласных равно количеству согласных и их порядковые номера; вывести слова в порядке убывания их длин

Листинг основного кода программы:

```
"""Laboratory Work No. 4.

Task 2. Regular expressions and text analysis.

Variant 13:
Text analyzer:
-Get a list of hexadecimal numbers contained in the text.
-Check if there are numbers in the text followed by a "+".
- Determine the number of words that are 4 characters long;
```

```

- find words with the number of vowels equal to the number of consonants and their ordinal
numbers;
- output words in descending order of their length

Version: 1.5
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-04-11
"""

# Исходные данные - из файла.
# Получение инфы через регулярки:
# - количество предложений в тексте;
# - количество предложений в тексте каждого вида отдельно (повествовательные, вопросительные
и побудительные);
# - среднюю длину предложения в символах (считаются только слова);
# - среднюю длину слова в тексте в символах;
# - количество смайликов в заданном тексте.

# Получить список шестнадцатеричных чисел, содержащихся в тексте.
# Проверить, существуют ли в тексте цифры, за которыми стоит «+».
# Определить число слов, длина которых равна 4 символа;
# найти слова, у которых количество гласных равно количеству согласных и их порядковые
номера;
# вывести слова в порядке убывания их длин.

# Вывести инфу на экран и сохранить в другой файл.
# Заархивировать файл с результатом с помощью модуля zipfile и обеспечить получение
информации о файле в архиве.

# слова = hex

from __future__ import annotations

import re
import zipfile
from pathlib import Path

from utils.input_utils import ask_yes_no

OUTPUT_DIR = Path("output")

# Источник
SOURCE_FILE = OUTPUT_DIR / "task2_source_text.txt"

# Файл результата, создается и перезаписывается авто

```

```

RESULT_FILE = OUTPUT_DIR / "task2_result.txt"

# ZIP-архив результата, тоже создается и перезаписывается программой
ARCHIVE_FILE = OUTPUT_DIR / "task2_result_archive.zip"

# Нех (обычные слова (bad, face, cafe) НЕ hex)
# Явно оформленные hex-записи:
# 1) 0xFF, 0X10
# 2) AB12, 2A3, 12AF – и цифра, и буква A-F
# 3) FACEh, BEEFh, 73h – суффикс h/H
HEX_PATTERN = (
    # группа без запоминания (чтобы объединить несколько вариантов, но не возвращать саму
    # группу отдельно)
    r"\b(?:"
    r"0[xX][0-9A-Fa-f]+"
    # ?= это первая проверка вперед : дальше должна быть хотя бы одна буква A-F или a-f
    r"|(?=[0-9A-Fa-f]*[A-Fa-f])(?=[0-9A-Fa-f]*\d)[0-9A-Fa-f]+[hH]?"
    r"| [0-9A-Fa-f]+[hH]"
    r")\b"
)

# Проверка цифр/чисел, за которыми стоит пробел и плюс.
# Пример: "(3 + 5) - 9x4" найдено "3".
# В выражении "17+20" ничего не будет найдено, потому что нет пробела перед плюсом.
DIGIT_BEFORE_PLUS_PATTERN = r"\b\d+(?=\s+\s+)"

# Слово:
# 1) обычная последовательность русских или английских букв;
# 2) явно оформленное шестнадцатеричное число.
# Чистые числа 7, 73, 2024 не считаются.
# Выражения 9x4 и 9x4 не считаются.
WORD_PATTERN = (
    r"\b(?:"
    r"0[xX][0-9A-Fa-f]+"
    r"|(?=[0-9A-Fa-f]*[A-Fa-f])(?=[0-9A-Fa-f]*\d)[0-9A-Fa-f]+[hH]?"
    r"| [0-9A-Fa-f]+[hH]"
    r"| [A-Za-zA-Яa-яЁё]+"
    r")\b"
)

# Предложение – фрагмент текста, который заканчивается одним или несколькими
# знаками конца предложения: ".", "?", "!".
# Поэтому "!!" – это конец предложения, "???" – тоже,
# а "?!?" – одновременно вопросительным и побудительным.
SENTENCE_PATTERN = r"^[.!?]+[.!?]+"

# Знаки конца предложения в самом конце найденного предложения.
ENDING_PATTERN = r"[.!?]+$"

```

```

# Смайлик по условию:
# 1) первым символом ровно один раз идет ";" или ":";
# 2) затем может идти "-" сколько угодно раз, даже 0;
# 3) затем обязательно идет одна или более одинаковых скобок:
#    "(", ")", "[", "]";
# 4) внутри смайлика нет других символов.
SMILE_PATTERN = (
    # < проверка смотрит назад, то есть на символы перед текущей позицией.
    # ! – отрицание, то есть не должно быть.
    r"(?![:;])"
    r"(?:[:;]|-)"
    r"_*"
    r"(?P<bracket>[\(\)\[\]])"
    r"(?P=bracket)*"
    r"(?![\(\)\[\];:-])"
)

# Гласные для русского и английского языков.
VOWELS = set("аеёиоуыёяАЕЁИОУЫЁЮАЕИОУ")

def read_source_text() -> str | None:
    """Read source text from the file."""
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    if not SOURCE_FILE.exists():
        print(f"\nSource file was not found: {SOURCE_FILE}")
        print("Create this file manually in VS Code and write text into it.")
        print("Then run task 2 again.")
        return None

    return SOURCE_FILE.read_text(encoding="utf-8")

def get_words(text: str) -> list[str]:
    """Return all words from text."""
    # re.findall возвращает все непересекающиеся совпадения.
    return re.findall(WORD_PATTERN, text)

# Счет гласных и согласных в 1 слове
def count_vowels_and_consonants(word: str) -> tuple[int, int]:
    """Count vowels and consonants in one word."""
    vowels_count = 0
    consonants_count = 0

    for char in word:
        if char in VOWELS:
            vowels_count += 1

```

```

        elif char.isalpha():
            consonants_count += 1

    return vowels_count, consonants_count

# Одинаковое кол-во гласных и согласных
def find_words_with_equal_vowels_and_consonants(text: str) -> list[tuple[int, str]]:
    """Find words where vowels count equals consonants count."""
    result = []
    words = get_words(text)

    for index, word in enumerate(words, start=1):
        vowels_count, consonants_count = count_vowels_and_consonants(word)

        # vowels_count > 0 нужно, чтобы не считать слова без гласных.
        # hex "FF" не должен попадать как слово с 0 гласных и 0 согласных.
        if vowels_count == consonants_count and vowels_count > 0:
            result.append((index, word))

    return result

def analyze_sentences(text: str) -> dict[str, float | int]:
    """Analyze sentence statistics."""
    sentences = re.findall(SENTENCE_PATTERN, text)

    narrative_count = 0
    question_count = 0
    imperative_count = 0

    # Здесь будут храниться количества слов в каждом предложении.
    # [5, 3, 7]: в первом предложении 5 слов, во втором 3, в третьем 7.
    words_count_in_sentences = []

    # Здесь будут храниться длины всех слов текста (слово "tree" даст длину 4)
    word_lengths = []

    for sentence in sentences:
        clear_sentence = sentence.strip()

        # Ищем знаки конца предложения: ".", "!", "!!", "?", "??", "?!".
        ending_match = re.search(ENDING_PATTERN, clear_sentence)

        if ending_match:
            ending = ending_match.group(0)

            # Повествовательное: заканчивается точкой и не содержит вопросительного или
            восклицательного знака.
            if "." in ending and "?" not in ending and "!" not in ending:

```

```

        narrative_count += 1

    # Вопросительное: в конце есть вопросительный знак.
    # "?" и "?! " считается вопросительным.
    if "?" in ending:
        question_count += 1

    # Побудительное: в конце есть восклицательный знак.
    # "!!" и "?! " считается побудительным.
    if "!" in ending:
        imperative_count += 1

    # Получаем слова конкретного предложения.
    words_in_sentence = get_words(clear_sentence)

    # Для средней длины предложения считаем количество слов в этом предложении.
    words_count_in_sentences.append(len(words_in_sentence))

    # Для средней длины слова сохраняем длину каждого найденного слова.
    for word in words_in_sentence:
        word_lengths.append(len(word))

# Средняя длина предложения: сумма слов по предложениям / количество предложений.
average_sentence_length = (
    sum(words_count_in_sentences) / len(words_count_in_sentences)
    if words_count_in_sentences
    else 0
)

# Средняя длина слова: сумма длин всех слов / количество слов.
average_word_length = (
    sum(word_lengths) / len(word_lengths)
    if word_lengths
    else 0
)

return {
    "total_sentences": len(sentences),
    "narrative_sentences": narrative_count,
    "question_sentences": question_count,
    "imperative_sentences": imperative_count,
    "average_sentence_length_in_words": round(average_sentence_length, 2),
    "average_word_length_in_chars": round(average_word_length, 2),
}

def find_smiles(text: str) -> list[str]:
    """Find full smile strings in text."""
    # Используем finditer, потому что в регулярке есть именованная группа.
    # match.group(0) возвращает полное найденное совпадение.

```

```

return [match.group(0) for match in re.finditer(SMILE_PATTERN, text)]

def build_report(text: str) -> str:
    """Build final text analysis report."""
    # 1. Список шестнадцатеричных чисел.
    hex_numbers = re.findall(HEX_PATTERN, text)

    # 2. Проверка, есть ли число, после которого стоит пробел и плюс.
    # search нужен для ответа "существует / не существует".
    digit_before_plus_exists = re.search(DIGIT_BEFORE_PLUS_PATTERN, text) is not None

    # findall нужен, чтобы вывести все такие числа.
    digits_before_plus = re.findall(DIGIT_BEFORE_PLUS_PATTERN, text)

    # 3. Все слова по правилу задачи.
    words = get_words(text)

    # 4. Слова длиной 4 символа.
    four_letter_words = [word for word in words if len(word) == 4]

    # 5. Слова, у которых количество гласных равно количеству согласных.
    equal_vowel_consonant_words = find_words_with_equal_vowels_and_consonants(text)

    # 6. Слова в порядке убывания длины.
    words_sorted_by_length = sorted(
        set(words),
        key=lambda word: (-len(word), word.lower()),
    )

    # 7. Анализ предложений.
    sentence_stats = analyze_sentences(text)

    # 8. Смайлики.
    smiles = find_smiles(text)

    # re.sub – замена: все hex-числа заменяем на три буквы HEX.
    text_after_sub = re.sub(HEX_PATTERN, "HEX", text)

    # re.split – разделение текста по знакам конца предложения.
    split_sentences = [
        part.strip()
        for part in re.split(r"[.!?]+", text)
        if part.strip()
    ]

    lines = [
        "Task 2. Regular expression analysis",
        "Developer: Kruchonok Aleksandra Sergeevna",
    ]

```

```

        "Development date: 2026-04-11",
        "=" * 80,
        "\nSource text:",
        text,
        "\n1. Hexadecimal numbers:",
        str(hex_numbers),
        "\n2. Numbers followed by space and plus:",
        str(digits_before_plus),
        f"Exist: {'yes' if digit_before_plus_exists else 'no'}",
        "\n3. Words with length equal to 4:",
        str(four_letter_words),
        f"Count: {len(four_letter_words)}",
        "\n4. Words where vowels count equals consonants count:",
        str(equal_vowel_consonant_words),
        "\n5. Words sorted by length descending:",
        str(words_sorted_by_length),
        "\n6. Sentence statistics:",
    ]

    for key, value in sentence_stats.items():
        lines.append(f"{key}: {value}")

    lines.extend(
        [
            "\n7. Smiles:",
            str(smiles),
            f"Smile count: {len(smiles)}",
            "\nText after re.sub: hexadecimal numbers replaced with HEX:",
            text_after_sub,
            "\nSentences after re.split:",
            str(split_sentences),
        ]
    )

    return "\n".join(lines)

def save_report(report: str) -> None:
    """Save report to result file."""
    # write_text перезаписывает файл результата при каждом запуске.
    RESULT_FILE.write_text(report, encoding="utf-8")

def create_archive() -> None:
    """Create ZIP archive with result file and print archive information."""
    # mode="w" перезаписывает архив при каждом запуске.
    # ZIP_DEFLATED вкл сжатие, compresslevel=9 - макс степень сжатия.
    with zipfile.ZipFile(
        ARCHIVE_FILE,

```



```

        mode="w",
        compression=zipfile.ZIP_DEFLATED,
        compresslevel=9,
    ) as archive:
        # добавление файла
        archive.write(RESULT_FILE, arcname=RESULT_FILE.name)

# getinfo получает информацию о файле внутри архива без извлечения.
with zipfile.ZipFile(ARCHIVE_FILE, mode="r") as archive:
    file_info = archive.getinfo(RESULT_FILE.name)

    original_size = file_info.file_size
    compressed_size = file_info.compress_size

    compression_percent = (
        (1 - compressed_size / original_size) * 100
        if original_size
        else 0
    )

    print("\nArchive information:")
    print(f"Archive: {ARCHIVE_FILE}")
    print(f"File in archive: {file_info.filename}")
    print(f"Original size: {original_size} bytes")
    print(f"Compressed size: {compressed_size} bytes")
    print(f"Compression percent: {compression_percent:.2f}%")
    print(f"Compression method code: {file_info.compress_type}")

def run_task_2() -> None:
    """Run task 2."""
    while True:
        print("\nTask 2. Regular expressions.")
        print("Developer: Kruchonok Aleksandra Sergeevna")
        print("Development date: 2026-04-11")

        text = read_source_text()

        if text is None:
            if not ask_yes_no("\nRepeat task 2? (y/n): "):
                break
            continue

        report = build_report(text)
        save_report(report)

        print("\n" + report)

        create_archive()

```

```

print(f"\nSource file: {SOURCE_FILE}")
print(f"Result file: {RESULT_FILE}")
print(f"Archive file: {ARCHIVE_FILE}")

if not ask_yes_no("\nRepeat task 2? (y/n): "):
    break

```

Работа программы (текст файла необходимо менять в самом файле):

```

=====
Choose task number: 2

Task 2. Regular expressions.
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-04-11

Task 2. Regular expression analysis
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-04-11
=====

Source text:
Today we export AB12, fAceh and 0xFF values. Another code is FACE and bad example is 12345.
Expression (3 + 5) - 9x4 is correct for the task, but 2 * 9 - 6 x 5 is not correct.
Digits with plus: 3 + 5, 17+20, 8 + 1.
Words: tree code milk мама тест домик python data film book.
Smile examples: :) :---))) ;-----[[[[[[[ ;---] :( --] ;--
This is a narrative sentence. Is this a question?! Stop it!

1. Hexadecimal numbers:
['AB12', 'fAceh', '0xFF']

2. Numbers followed by space and plus:
['3', '3', '8']
Exist: yes

3. Words with length equal to 4:
['AB12', '0xFF', 'code', 'FACE', 'task', 'with', 'plus', 'tree', 'code', 'milk', 'мама', 'тест', 'd
ata', 'film', 'book', 'This', 'this', 'Stop']
Count: 18

4. Words where vowels count equals consonants count:
[(2, 'we'), (4, 'AB12'), (8, 'values'), (10, 'code'), (11, 'is'), (12, 'FACE'), (16, 'is'), (18, 'i
s'), (25, 'is'), (32, 'tree'), (33, 'code'), (35, 'мама'), (39, 'data'), (41, 'book'), (45, 'is'),
(49, 'Is'), (52, 'question'), (54, 'it')]

5. Words sorted by length descending:
['Expression', 'narrative', 'examples', 'question', 'sentence', 'Another', 'correct', 'example', 'D
igits', 'export', 'python', 'values', 'fAceh', 'Smile', 'Today', 'Words', 'домик', '0xFF', 'AB12',
'book', 'code', 'data', 'FACE', 'film', 'milk', 'plus', 'Stop', 'task', 'This', 'this', 'tree', 'wi
th', 'мама', 'тест', 'and', 'bad', 'but', 'for', 'not', 'the', 'is', 'Is', 'it', 'we', 'a', 'x']

```

```
6. Sentence statistics:
total_sentences: 8
narrative_sentences: 6
question_sentences: 1
imperative_sentences: 2
average_sentence_length_in_words: 6.75
average_word_length_in_chars: 4.3

7. Smiles:
[':)', ':(---))', ';-----[[[[[[[', ';---]', ':((']
Smile count: 5
```

```
Archive information:
Archive: output\task2_result_archive.zip
File in archive: task2_result.txt
Original size: 2911 bytes
Compressed size: 1034 bytes
Compression percent: 64.48%
Compression method code: 8

Source file: output\task2_source_text.txt
Result file: output\task2_result.txt
Archive file: output\task2_result_archive.zip

Repeat task 2? (y/n):
```

Дополнительная демонстрация методов re.sub и re.split

```
Text after re.sub: hexadecimal numbers replaced with HEX:
Today we export HEX, HEX and HEX values. Another code is FACE and bad example is 12345.
Expression (3 + 5) - 9x4 is correct for the task, but 2 * 9 - 6 x 5 is not correct.
Digits with plus: 3 + 5, 17+20, 8 + 1.
Words: tree code milk мама тест домик python data film book.
Smile examples: :) :---))) ;-----[[[[[[[ ;---] :( --] ;--
This is a narrative sentence. Is this a question?! Stop it!

Sentences after re.split:
['Today we export AB12, fAceh and 0xFF values', 'Another code is FACE and bad example is 12345', 'E
xpression (3 + 5) - 9x4 is correct for the task, but 2 * 9 - 6 x 5 is not correct', 'Digits with pl
us: 3 + 5, 17+20, 8 + 1', 'Words: tree code milk мама тест домик python data film book', 'Smile exa
mples: :) :---))) ;-----[[[[[[[ ;---] :( --] ;-- \nThis is a narrative sentence', 'Is this a qu
estion', 'Stop it']

Archive information:
Archive: output\task2_result_archive.zip
File in archive: task2_result.txt
Original size: 2911 bytes
Compressed size: 1034 bytes
Compression percent: 64.48%
Compression method code: 8

Source file: output\task2_source_text.txt
Result file: output\task2_result.txt
Archive file: output\task2_result_archive.zip

Repeat task 2? (y/n):
```

Задание 3. В соответствии с заданием своего варианта доработать программу из ЛР3, используя класс и обеспечить:

- а) определение дополнительных параметров среднее арифметическое элементов последовательности, медиана, мода, дисперсия, СКО последовательности;
- б) с помощью библиотеки matplotlib нарисовать графики разных цветов в одной координатной оси:

- график по полученным данным разложения функции в ряд, представленным в таблице,
- график соответствующей функции, представленной с помощью модуля math.

Обеспечить отображение координатных осей, легенды, текста и аннотации.

x	n	$F(x)$	$Math F(x)$	eps

Здесь x – значение аргумента, $F(x)$ – значение функции, n – количество просуммированных членов ряда, $Math F(x)$ – значение функции, вычисленное с помощью модуля math.

- в) сохранить графики в файл.

Вар-т	Условие
13	$\ln(1+x) = \sum_{n=0}^{\infty} (-1)^{n-1} \frac{x^n}{n} = x - \frac{x^2}{2} + \frac{x^3}{3} + \dots, x < 1$

Листинг основного кода программы:

```

"""Laboratory Work No. 4.

Task 3. ln(1 + x), class, statistics and matplotlib plot.

Variant 13:
ln(1 + x) = x - x^2 / 2 + x^3 / 3 - ...

Version: 1.3
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-04-14
"""

from __future__ import annotations

import math
import statistics
from collections import Counter
from pathlib import Path

import matplotlib.pyplot as plt

```

```

from utils.input_utils import ask_int_in_range, ask_positive_float, ask_yes_no

OUTPUT_DIR = Path("output")
PLOT_FILE = OUTPUT_DIR / "task3_ln_series_plot.png"

class LnSeriesExperiment:
    """Calculate  $\ln(1 + x)$  using a power series and analyze results."""

    MAX_ITERATIONS = 20000

    def __init__(self, eps: float) -> None:
        """Initialize experiment with selected precision."""
        # self – ссылка на текущий объект; eps хранится внутри объекта и используется всеми
        # методами класса.
        self.eps = eps

        # Данные таблицы хранятся в отдельных списках.
        self.table_x: list[float] = []
        self.table_terms: list[int] = []
        self.table_series_values: list[float] = []
        self.table_math_values: list[float] = []

    @staticmethod
    def is_valid_x(x: float) -> bool:
        """Check if x belongs to the convergence interval."""
        # Ряд  $\ln(1 + x)$  сходится при  $-1 < x \leq 1$ .
        # безопасный интервал  $-0.8 \dots 0.8$ .
        return -1 < x < 1

    def calculate_series(self, x: float) -> tuple[float, int]:
        """Calculate  $\ln(1 + x)$  using the power series."""
        if not self.is_valid_x(x):
            raise ValueError("x must satisfy  $-1 < x < 1$ .")

        # Формула ряда:  $\ln(1 + x) = x - x^2/2 + x^3/3 - x^4/4 + \dots$ 
        result = 0.0
        term = x
        n = 1

        # Суммируем члены ряда, пока модуль текущего члена  $\geq$  eps.
        while abs(term) >= self.eps and n <= self.MAX_ITERATIONS:
            result += term

            # Рекуррентная формула для следующего члена ряда.
            # Если текущий член:  $(-1)^{(n-1)} * x^n / n$ , то следующий получаем умножением на  $-x*n/(n+1)$ .

```

```

        n += 1
        term *= -x * (n - 1) / n

    terms_count = n - 1
    return result, terms_count

@staticmethod
def make_x_values(start: float, end: float, count: int) -> list[float]:
    """Create evenly spaced x values."""
    step = (end - start) / (count - 1)
    return [start + step * index for index in range(count)]

def build_table_points(self, start: float, end: float, count: int) -> None:
    """Build table values."""
    self.table_x.clear()
    self.table_terms.clear()
    self.table_series_values.clear()
    self.table_math_values.clear()

    x_values = self.make_x_values(start, end, count)

    for x in x_values:
        series_value, terms_count = self.calculate_series(x)
        math_value = math.log(1 + x)

        self.table_x.append(x)
        self.table_terms.append(terms_count)
        self.table_series_values.append(series_value)
        self.table_math_values.append(math_value)

def calculate_statistics(self) -> dict[str, float]:
    """Calculate mean, median, mode, variance and standard deviation."""
    values = self.table_series_values

    if not values:
        return {
            "mean": 0.0,
            "median": 0.0,
            "mode": 0.0,
            "variance": 0.0,
            "standard_deviation": 0.0,
        }

    # Среднее арифметическое – сумма значений / количество значений.
    mean_value = statistics.mean(values)

    # Медиана – центральное значение отсортированной последовательности.
    # Если элементов четное количество, берется среднее двух центральных.
    median_value = statistics.median(values)

```

```

# Мода – самое часто встречающееся значение.
# Для float-значений почти все числа разные, округление до 4 знаков.
rounded_values = [round(value, 4) for value in values]
mode_counter = Counter(rounded_values)
mode_value = mode_counter.most_common(1)[0][0]

# Дисперсия – среднее арифметическое квадратов отклонений от среднего.
variance_value = statistics.pvariance(values)

# СКО – стандартное квадратическое отклонение, квадратный корень из дисперсии.
standard_deviation_value = statistics.pstdev(values)

return {
    "mean": mean_value,
    "median": median_value,
    "mode": mode_value,
    "variance": variance_value,
    "standard_deviation": standard_deviation_value,
}

def print_table(self) -> None:
    """Print calculation table."""
    print("\n" + "-" * 95)
    print(
        f"{'x':>12} "
        f"{'n':>8} "
        f"{'Series F(x)':>20} "
        f"{'math.log(1+x)':>20} "
        f"{'Difference':>15}"
    )
    print("-" * 95)

    for x, n, series_value, math_value in zip(
        self.table_x,
        self.table_terms,
        self.table_series_values,
        self.table_math_values,
    ):
        difference = abs(series_value - math_value)

        print(
            f"{'x':>12.6f} "
            f"{'n':>8} "
            f"{'series_value':>20.10f} "
            f"{'math_value':>20.10f} "
            f"{'difference':>15.2e}"
        )

```

```

print("-" * 95)

@staticmethod
def draw_math_axes(
    ax,
    x_min: float,
    x_max: float,
    y_min: float,
    y_max: float,
) -> None:
    """Draw mathematical coordinate axes with arrows."""
    ax.spines[["left", "right", "top", "bottom"]].set_visible(False)

    # Ось Ox – горизонтальная стрелка.
    ax.annotate(
        "",
        xy=(x_max, 0),
        xytext=(x_min, 0),
        arrowprops={"arrowstyle": "->", "linewidth": 1.2, "color": "black"},
    )

    # Ось Oy – вертикальная стрелка.
    ax.annotate(
        "",
        xy=(0, y_max),
        xytext=(0, y_min),
        arrowprops={"arrowstyle": "->", "linewidth": 1.2, "color": "black"},
    )

    # Подпись Ox ставим справа после конца стрелки.
    ax.text(
        x_max,
        -0.04 * (y_max - y_min),
        "Ox",
        fontsize=12,
        ha="left",
        va="top",
        clip_on=False,
    )

    # Подпись Oy ставим сверху после конца стрелки.
    ax.text(
        0.03 * (x_max - x_min),
        y_max,
        "Oy",
        fontsize=12,
        ha="left",
        va="bottom",
        clip_on=False,
    )

```



```

)

def build_plot(self, filename: Path) -> None:
    """Build and save the plot."""
    filename.parent.mkdir(parents=True, exist_ok=True)

    # Для таблицы пользователь выбирает немного точек.
    # Для графика берем много точек, чтобы кривая была плавной, а не прямой.
    plot_x = self.make_x_values(-0.8, 0.8, 200)

    plot_series_values = [self.calculate_series(x)[0] for x in plot_x]
    plot_math_values = [math.log(1 + x) for x in plot_x]

    all_y_values = plot_series_values + plot_math_values

    x_min = min(plot_x) - 0.1
    x_max = max(plot_x) + 0.1
    y_min = min(all_y_values) - 0.2
    y_max = max(all_y_values) + 0.2

    fig, ax = plt.subplots(figsize=(10, 6))

    ax.set_xlim(x_min, x_max)
    ax.set_ylim(y_min, y_max)

    # Первый график – значения, рассчитанные через ряд.
    ax.plot(
        plot_x,
        plot_series_values,
        linewidth=2,
        label="Power series F(x)",
    )

    # Второй график – значения через math.log.
    ax.plot(
        plot_x,
        plot_math_values,
        linewidth=2,
        linestyle="--",
        label="math.log(1 + x)",
    )

    # Рисуем математические оси Ox и Oy со стрелками.
    self.draw_math_axes(ax, x_min, x_max, y_min, y_max)

    # Сетка помогает визуально читать значения.
    ax.grid(True)

    # Заголовок поднят выше через pad.

```

```

ax.set_title("Graph of  $\ln(1 + x)$ ", fontsize=15, pad=28)

# Легенда формируется из label в ax.plot(). bbox_to_anchor переносит легенду вниз под
график.
ax.legend(
    loc="upper center",
    bbox_to_anchor=(0.5, -0.12),
    ncol=2,
)

# Оставляем место снизу под легенду и сверху под заголовок.
fig.subplots_adjust(bottom=0.22, top=0.82)

# Сохранение графика в файл.
# dpi=300 задает хорошее качество изображения. bbox_inches="tight" обрезает лишние
поля.
plt.savefig(filename, dpi=300, bbox_inches="tight")

# Показываем график на экране.
plt.show()

def run_task_3() -> None:
    """Run task 3."""
    while True:
        print("\nTask 3.  $\ln(1 + x)$ , statistics and matplotlib.")
        print("Developer: Kruchonok Aleksandra Sergeevna")
        print("Development date: 2026-04-14")

        print("\nFor the series  $\ln(1 + x)$ , use interval  $-1 < x < 1$ .")
        eps = ask_positive_float("Enter eps, for example 0.0001: ")

        print("\nThe program uses a safe interval from -0.8 to 0.8.")
        count = ask_int_in_range("Enter number of table points from 3 to 30: ", 3, 30)

        experiment = LnSeriesExperiment(eps=eps)

        # type показывает тип объекта.
        print(f"\nObject type: {type(experiment)}")

        experiment.build_table_points(start=-0.8, end=0.8, count=count)

        experiment.print_table()

        statistics_result = experiment.calculate_statistics()

        print("\nAdditional statistical parameters for  $F(x)$ :")
        print(f"Mean: {statistics_result['mean']:.10f}")
        print(f"Median: {statistics_result['median']:.10f}")

```

```

print(f"Mode rounded to 4 decimals: {statistics_result['mode']:.4f}")
print(f"Variance: {statistics_result['variance']:.10f}")
print(f"Standard deviation: {statistics_result['standard_deviation']:.10f}")

experiment.build_plot(PLOT_FILE)
print(f"\nPlot saved to: {PLOT_FILE}")

if not ask_yes_no("\nRepeat task 3? (y/n): "):
    break

```

Работа программы (ввод eps (0.22) и количество значений для таблицы (10)):

```

=====
Choose task number: 3

```

```

Task 3.  $\ln(1 + x)$ , statistics and matplotlib.
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-04-14

```

```

For the series  $\ln(1 + x)$ , use interval  $-1 < x < 1$ .
Enter eps, for example 0.0001: 0.22

```

```

The program uses a safe interval from -0.8 to 0.8.
Enter number of table points from 3 to 30: 10

```

```

Object type: <class 'tasks.task3_plots.LnSeriesExperiment'>

```

x	n	Series F(x)	math.log(1+x)	Difference
-0.800000	2	-1.1200000000	-1.6094379124	4.89e-01
-0.622222	1	-0.6222222222	-0.9734491457	3.51e-01
-0.444444	1	-0.4444444444	-0.5877866649	1.43e-01
-0.266667	1	-0.2666666667	-0.3101549283	4.35e-02
-0.088889	0	0.0000000000	-0.0930904231	9.31e-02
0.088889	0	0.0000000000	0.0851578083	8.52e-02
0.266667	1	0.2666666667	0.2363887781	3.03e-02
0.444444	1	0.4444444444	0.3677247801	7.67e-02
0.622222	1	0.6222222222	0.4837969514	1.38e-01
0.800000	2	0.4800000000	0.5877866649	1.08e-01

Additional statistical parameters for F(x):

Mean: -0.0640000000

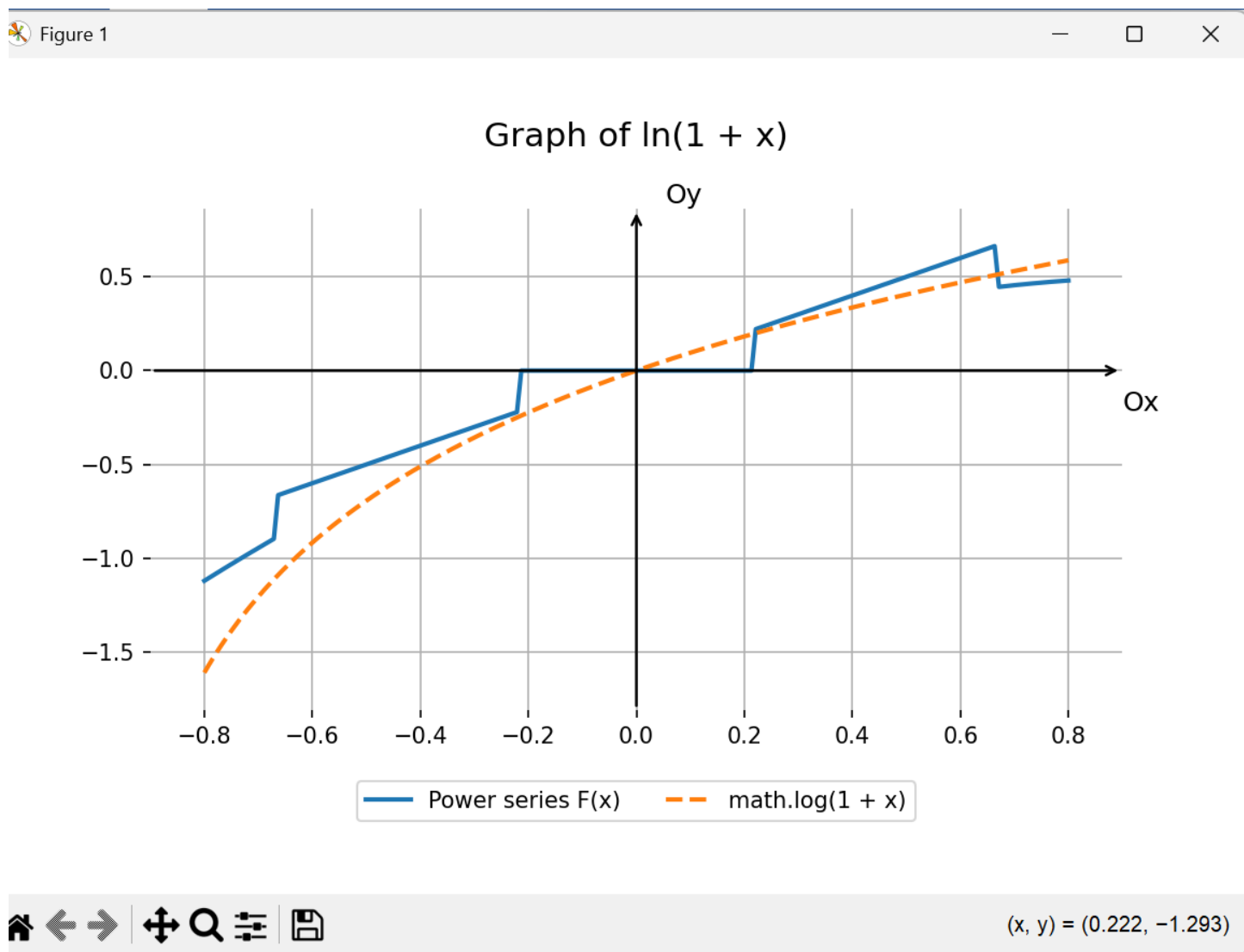
Median: 0.0000000000

Mode rounded to 4 decimals: 0.0000

Variance: 0.2755444938

Standard deviation: 0.5249233219

График:



Задание 4. В соответствии с заданием своего варианта разработать базовые классы и классы наследники.

Требования по использованию классов:

Абстрактный класс «**Геометрическая фигура**» содержит абстрактный метод для вычисления площади фигуры (<https://docs.python.org/3/library/abc.html>) .

Класс «**Цвет фигуры**» содержит свойство для описания цвета геометрической фигуры (<https://docs.python.org/3/library/functions.html#property>) .

Класс «**Прямоугольник**» (Круг, Ромб, Квадрат, Треугольник и т.д.) наследуется от класса «Геометрическая фигура». Класс должен содержать конструктор по параметрам «ширина», «высота» (для другого типа фигуры соответствующие параметры, например, для круга задаем «радиус») и «цвет». В конструкторе создается объект класса «Цвет фигуры» для хранения цвета. Класс должен переопределять метод, вычисляющий площадь фигуры <https://docs.python.org/3/library/math.html> .

Для класса «**Прямоугольник**» (тип фигуры в инд. задании) определить метод, который возвращает в виде строки основные параметры фигуры, ее цвет и площадь. Использовать метод format (<https://pyformat.info/>) название фигуры должно задаваться в виде поля данных класса и возвращаться методом класса. В корневом каталоге проекта создайте файл main.py для тестирования классов.

Используйте конструкцию, описанную в https://docs.python.org/3/library/_main_.html

Пример объекта: Прямоугольник синего цвета шириной 5 и высотой 8. Программа должна содержать следующие базовые функции:

- 1) ввод значений параметров пользователем;
- 2) проверка корректности вводимых данных;
- 3) построение, закрашивание фигуры в выбранный цвет, введенный с клавиатуры, и подпись фигуры текстом, введенным с клавиатуры;
- 4) вывод фигуры на экран и в файл.

Вар-т	Условие
13	Построить треугольник со сторонами a, b, c.

Листинг основного кода программы:

```
"""Laboratory Work No. 4.

Task 4. Triangle class hierarchy.

Variant 13:
Build a triangle with sides a, b, c.

Version: 2.3
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-04-17
```

```

"""

from __future__ import annotations

import math
from abc import ABC, abstractmethod
from pathlib import Path

import matplotlib.pyplot as plt

from utils.input_utils import ask_non_empty_string, ask_positive_float, ask_yes_no

OUTPUT_DIR = Path("output")
TRIANGLE_IMAGE = OUTPUT_DIR / "task4_triangle.png"
TRIANGLE_INFO = OUTPUT_DIR / "task4_triangle_info.txt"

class GeometricFigure(ABC):
    """Abstract base class for geometric figures."""

    # Атрибут класса, принадлежит классу.
    figure_type = "Geometric figure"

    def __init__(self, name: str) -> None:
        """Initialize a geometric figure."""
        # self — ссылка на текущий объект.
        # name — динамический атрибут объекта, создается во время __init__.
        self.name = name

    @abstractmethod
    def calculate_area(self) -> float:
        """Calculate figure area."""

    @classmethod
    def get_figure_type(cls) -> str:
        """Return figure type."""
        # cls — ссылка на класс.
        # вызывается через имя класса: Triangle.get_figure_type().
        return cls.figure_type

    @staticmethod
    def is_positive(value: float) -> bool:
        """Check whether value is positive."""
        # Статический метод не использует ни self, ни cls. Он логически относится к классу.
        return value > 0

```

```

class FigureColor:
    """Store and validate a figure color."""

    ALLOWED_COLORS = {
        "red",
        "green",
        "blue",
        "yellow",
        "orange",
        "purple",
        "black",
        "white",
        "pink",
        "gray",
        "cyan",
    }

    def __init__(self, color: str) -> None:
        """Initialize color storage."""
        # __color – private-атрибут.
        # Снаружи напрямую к нему обращаться не нужно.
        self.__color = "blue"

        # Через property-сеттер сразу проверяем корректность цвета.
        self.color = color

    @property
    def color(self) -> str:
        """Get figure color."""
        # Getter возвращает значение private-атрибута.
        return self.__color

    @color.setter
    def color(self, value: str) -> None:
        """Set figure color."""
        # Setter проверяет значение перед записью.
        normalized = value.strip().lower()

        if not normalized:
            raise ValueError("Color cannot be empty.")

        if normalized not in self.ALLOWED_COLORS:
            raise ValueError(
                f"Unsupported color: {value}. "
                f"Use one of: {' '.join(sorted(self.ALLOWED_COLORS))}."
            )

        self.__color = normalized

```

```

class PerimeterMixin:
    """Mixin that adds perimeter functionality."""

    def calculate_perimeter(self) -> float:
        """Calculate figure perimeter."""
        # Mixin использует _sides. protected-атрибут, у класса Triangle.
        return sum(self._sides)

    def perimeter_text(self) -> str:
        """Return formatted perimeter text."""
        return f"Perimeter: {self.calculate_perimeter():.4f}"

class Triangle(PerimeterMixin, GeometricFigure):
    """Represent a triangle with sides a, b and c."""

    # Переопределяем атрибут класса базового класса.
    figure_type = "Triangle"

    def __init__(self, a: float, b: float, c: float, color: str, text: str) -> None:
        """Initialize a triangle."""
        # super() вызывает __init__ родительского класса GeometricFigure, базовая часть
        # объекта получает имя "Triangle".
        super().__init__(name="Triangle")

        if not self.is_valid_sides(a, b, c):
            raise ValueError("Triangle sides are invalid.")

        # Одно подчеркивание – protected.
        # атрибут для внутреннего использования и для использования в наследниках.
        self._sides = [a, b, c]

        # Два подчеркивания – private через name mangling.
        # Python переименует __label примерно в _Triangle__label.
        self.__label = text

        # Композиция: внутри Triangle хранится объект FigureColor.
        # Triangle не наследуется от FigureColor, а использует его.
        self._figure_color = FigureColor(color)

    @staticmethod
    def is_valid_sides(a: float, b: float, c: float) -> bool:
        """Check triangle inequality."""
        return (
            a > 0
            and b > 0
            and c > 0
            and a + b > c
            and a + c > b

```



```

        and b + c > a
    )

    @property
    def a(self) -> float:
        """Get first side."""
        return self._sides[0]

    @property
    def b(self) -> float:
        """Get second side."""
        return self._sides[1]

    @property
    def c(self) -> float:
        """Get third side."""
        return self._sides[2]

    @property
    def color(self) -> str:
        """Get triangle color."""
        return self._figure_color.color

    @color.setter
    def color(self, value: str) -> None:
        """Set triangle color."""
        self._figure_color.color = value

    @property
    def label(self) -> str:
        """Get triangle label."""
        return self.__label

    @label.setter
    def label(self, value: str) -> None:
        """Set triangle label."""
        normalized = value.strip()

        if not normalized:
            raise ValueError("Label cannot be empty.")

        self.__label = normalized

    def calculate_area(self) -> float:
        """Calculate triangle area using Heron's formula."""
        # Формула Герона:
        #  $p = (a + b + c) / 2$ 
        #  $S = \sqrt{p * (p-a) * (p-b) * (p-c)}$ 
        semi_perimeter = self.calculate_perimeter() / 2

```

```

        return math.sqrt(
            semi_perimeter
            * (semi_perimeter - self.a)
            * (semi_perimeter - self.b)
            * (semi_perimeter - self.c)
        )

def get_coordinates(self) -> tuple[list[float], list[float]]:
    """Calculate triangle coordinates for drawing."""
    # Размещаем сторону c на оси Ox: A = (0, 0), B = (c, 0).
    # Координаты C считаем из длин сторон.
    x_c = (self.b ** 2 + self.c ** 2 - self.a ** 2) / (2 * self.c)
    y_c = math.sqrt(max(self.b ** 2 - x_c ** 2, 0))

    # Последняя точка повторяет первую, чтобы линия замкнулась.
    x_values = [0, self.c, x_c, 0]
    y_values = [0, 0, y_c, 0]

    return x_values, y_values

def get_info(self) -> str:
    """Return formatted triangle information."""
    return (
        f"{self.name}\n"
        f"a = {self.a:.2f}\n"
        f"b = {self.b:.2f}\n"
        f"c = {self.c:.2f}\n"
        f"color = {self.color}\n"
        f"area = {self.calculate_area():.4f}\n"
        f"{self.perimeter_text()}"
    )

@staticmethod
def draw_math_axes(
    ax,
    x_min: float,
    x_max: float,
    y_min: float,
    y_max: float,
) -> None:
    """Draw mathematical coordinate axes with arrows."""
    # Убираем стандартную рамку matplotlib.
    ax.spines[["left", "right", "top", "bottom"]].set_visible(False)

    # Ось Ox – горизонтальная стрелка.
    ax.annotate(
        "",
        xy=(x_max, 0),

```

```

        xytext=(x_min, 0),
        arrowprops={"arrowstyle": "->", "linewidth": 1.2, "color": "black"},
    )

    # Ось Oy – вертикальная стрелка.
    ax.annotate(
        "",
        xy=(0, y_max),
        xytext=(0, y_min),
        arrowprops={"arrowstyle": "->", "linewidth": 1.2, "color": "black"},
    )

    # Подпись Ox справа после конца стрелки.
    ax.text(
        x_max,
        -0.05 * (y_max - y_min),
        "Ox",
        fontsize=12,
        ha="left",
        va="top",
        clip_on=False,
    )

    # Подпись Oy сверху после конца стрелки.
    ax.text(
        0.03 * (x_max - x_min),
        y_max,
        "Oy",
        fontsize=12,
        ha="left",
        va="bottom",
        clip_on=False,
    )

def draw(self, image_path: Path, info_path: Path) -> None:
    """Draw triangle and save it to files."""
    image_path.parent.mkdir(parents=True, exist_ok=True)

    x_values, y_values = self.get_coordinates()

    max_x = max(x_values)
    max_y = max(y_values)

    # Оставляем справа место для подписи в рамке.
    x_min = -0.5
    x_max = max_x + 3.2
    y_min = -0.5
    y_max = max_y + 0.8

```

```

fig, ax = plt.subplots(figsize=(9, 6))

ax.set_xlim(x_min, x_max)
ax.set_ylim(y_min, y_max)

# Заливка треугольника.
ax.fill(
    x_values,
    y_values,
    color=self.color,
    alpha=0.35,
    label="Triangle area",
)

# Контур треугольника.
ax.plot(
    x_values,
    y_values,
    marker="o",
    linewidth=2,
    label="Triangle border",
)

# Математические оси Ox/Oy.
self.draw_math_axes(ax, x_min, x_max, y_min, y_max)

# Подпись фигуры в рамке справа от фигурки
ax.text(
    max_x + 0.35,
    max_y * 0.55 if max_y else 0.5,
    f"{self.label}\n\n{self.get_info()}",
    fontsize=10,
    ha="left",
    va="center",
    bbox={
        "boxstyle": "round",
        "facecolor": "white",
        "edgecolor": "black",
        "alpha": 0.9,
    },
)

# Заголовок графика.
ax.set_title("Triangle with sides a, b, c", fontsize=15, pad=28)

# Сетка.
ax.grid(True)

# Сохраняем одинаковый масштаб по осям, чтобы фигура не искажалась.

```

```

ax.set_aspect("equal", adjustable="box")

# Легенда формируется из label в ax.fill и ax.plot.
ax.legend(
    loc="upper center",
    bbox_to_anchor=(0.5, -0.10),
    ncol=2,
)

# Оставляем место под легенду и заголовок.
fig.subplots_adjust(bottom=0.20, top=0.82)

# Сохранение картинки.
plt.savefig(image_path, dpi=300, bbox_inches="tight")
plt.show()

# Сохранение текстовой информации о фигуре.
info_path.write_text(self.get_info(), encoding="utf-8")

def __str__(self) -> str:
    """Return readable string representation."""
    # print(triangle) вызывает именно __str__.
    return self.get_info()

def __repr__(self) -> str:
    """Return technical string representation."""
    return (
        f"Triangle(a={self.a!r}, b={self.b!r}, "
        f"c={self.c!r}, color={self.color!r}, label={self.label!r})"
    )

def __len__(self) -> int:
    """Return number of triangle sides."""
    # Позволяет вызвать len(triangle).
    return 3

def __getitem__(self, index: int) -> float:
    """Get side by index."""
    # Позволяет обращаться к сторонам по индексу: triangle[0], triangle[1], triangle[2].
    return self._sides[index]

def __eq__(self, other: object) -> bool:
    """Compare triangles by area."""
    # Переопределение оператора ==.
    if not isinstance(other, Triangle):
        return NotImplemented

    return math.isclose(self.calculate_area(), other.calculate_area())

```

```

def __lt__(self, other: "Triangle") -> bool:
    """Compare triangles by area."""
    # Переопределение оператора <.
    return self.calculate_area() < other.calculate_area()

def ask_color() -> str:
    """Read correct matplotlib color."""
    while True:
        color = ask_non_empty_string(
            "Enter color in English "
            "(red/green/blue/yellow/orange/purple/black/white/pink/gray/cyan): "
        )

        try:
            FigureColor(color)
            return color
        except ValueError as error:
            print(error)

def run_task_4() -> None:
    """Run task 4."""
    while True:
        print("\nTask 4. Triangle class hierarchy.")
        print("Developer: Kruchonok Aleksandra Sergeevna")
        print("Development date: 2026-04-17")

        while True:
            a = ask_positive_float("Enter side a: ")
            b = ask_positive_float("Enter side b: ")
            c = ask_positive_float("Enter side c: ")

            if Triangle.is_valid_sides(a, b, c):
                break

        print("Input error: these sides cannot form a triangle.")

        color = ask_color()
        label = ask_non_empty_string("Enter triangle label text: ")

        triangle = Triangle(a=a, b=b, c=c, color=color, text=label)

        print("\nTriangle created:")
        print(triangle)

        print("\nClass and object examples:")
        print(f"type(triangle): {type(triangle)}")
        print(f"Class method Triangle.get_figure_type(): {Triangle.get_figure_type()}")

```

```

    print(f"Static method Triangle.is_valid_sides(a, b, c): {Triangle.is_valid_sides(a,
b, c)}")

    print("\nMagic methods examples:")
    print(f"str(triangle): {str(triangle)}")
    print(f"repr(triangle): {repr(triangle)}")
    print(f"len(triangle): {len(triangle)}")
    print(f"triangle[0]: {triangle[0]}")
    print(f"triangle[1]: {triangle[1]}")
    print(f"triangle[2]: {triangle[2]}")

    print("\nEncapsulation examples:")
    print(f"Protected field triangle._sides: {triangle._sides}")
    print(f"Private label through property triangle.label: {triangle.label}")
    print("Private technical access: triangle._Triangle__label")

    triangle.draw(TRIANGLE_IMAGE, TRIANGLE_INFO)

    print(f"\nTriangle image saved to: {TRIANGLE_IMAGE}")
    print(f"Triangle information saved to: {TRIANGLE_INFO}")

    if not ask_yes_no("\nRepeat task 4? (y/n): "):
        break

```

Работа программы (ввод сторон, цвета и подписи):

```

=====
Choose task number: 4

Task 4. Triangle class hierarchy.
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-04-17
Enter side a: 3
Enter side b: 4
Enter side c: 5
Enter color in English (red/green/blue/yellow/orange/purple/black/white/pink/gray/cyan): yellow
Enter triangle label text: Шедевр

```

```

Triangle created:
Triangle
a = 3.00
b = 4.00
c = 5.00
color = yellow
area = 6.0000
Perimeter: 12.0000

Class and object examples:
type(triangle): <class 'tasks.task4_triangle.Triangle'>
Class method Triangle.get_figure_type(): Triangle
Static method Triangle.is_valid_sides(a, b, c): True

```

Демонстрация основных поинтов задания для защиты:

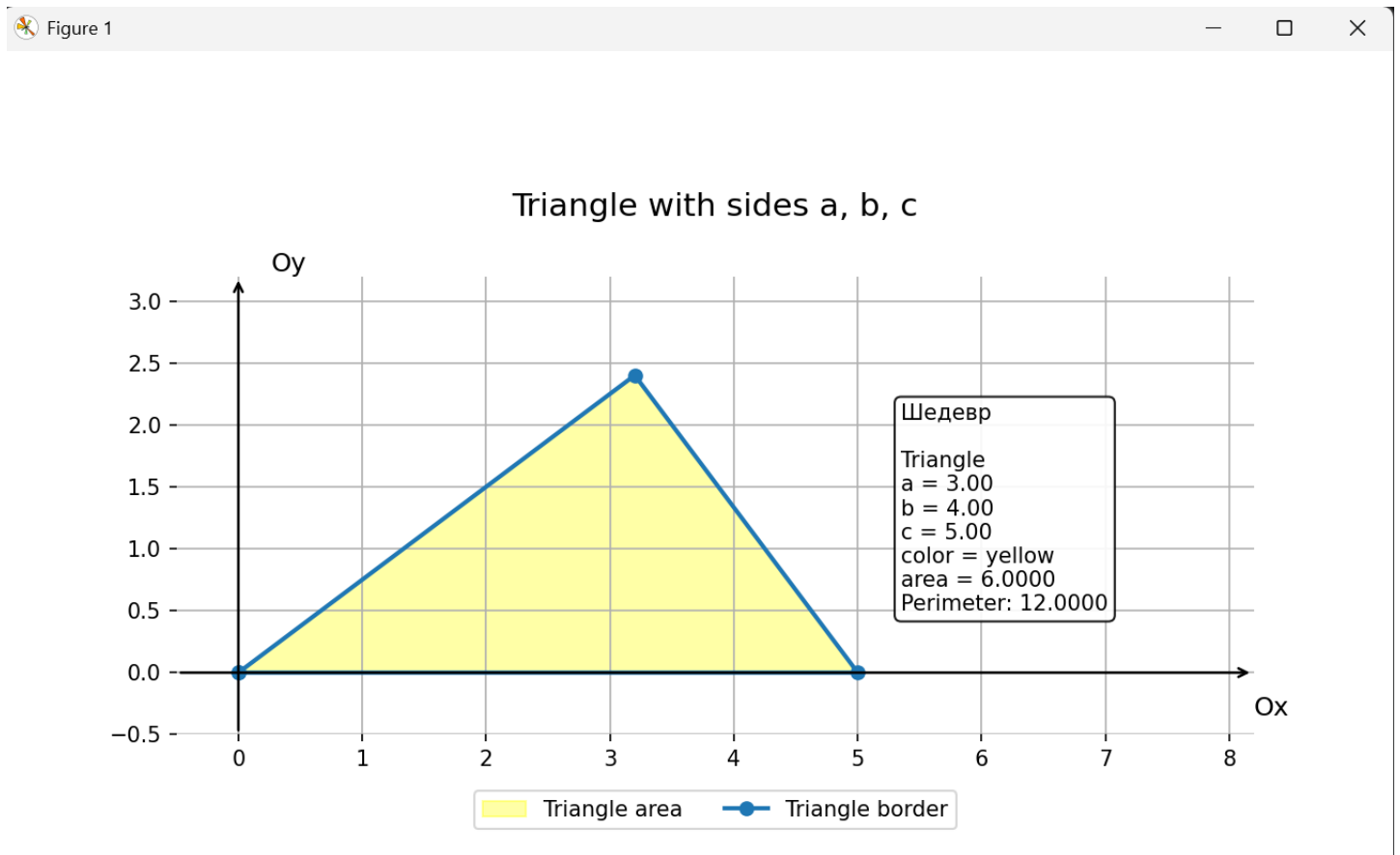
```

Magic methods examples:
str(triangle): Triangle
a = 3.00
b = 4.00
c = 5.00
color = yellow
area = 6.0000
Perimeter: 12.0000
repr(triangle): Triangle(a=3.0, b=4.0, c=5.0, color='yellow', label='Шедевр')
len(triangle): 3
triangle[0]: 3.0
triangle[1]: 4.0
triangle[2]: 5.0

Encapsulation examples:
Protected field triangle._sides: [3.0, 4.0, 5.0]
Private label through property triangle.label: Шедевр
Private technical access: triangle._Triangle__label

```

График:



Задание 5. В соответствии с заданием своего варианта исследовать возможности библиотеки NumPy при работе с массивами и математическими и статическими операциями.

Сформировать целочисленную матрицу $A[n,m]$ с помощью генератора случайных чисел (random).

а) Библиотека NumPy.

1. Создание массива. Функции `array()` и `values()`.
2. Функции создания массива заданного вида.
3. Индексирование массивов NumPy. Индекс и срез.
4. Операции с массивами. Универсальные (поэлементные) функции.

б) Математические и статистические операции.

1. Функция `mean()`
2. Функция `median()`
3. Функция `corrcoef()`
4. Дисперсия `var()`.
5. Стандартное отклонение `std()`

Вар-т	Условие
13	Получить новую матрицу путем деления всех элементов исходной матрицы на ее наибольший по модулю элемент. Вычислить дисперсию элементов новой матрицы. Ответ округлите до сотых. Вычисление дисперсии выполнить двумя способами: через стандартную функцию и через программирование формулы.

Листинг основного кода программы:

```
""" Laboratory Work No. 4.
Task 5. NumPy arrays, matrix initialization, indexing, slices, universal functions,
mathematical and statistical operations.

Variant 13:
Create a new matrix by dividing all elements of the source matrix by the maximum absolute
element.
Calculate variance of the new matrix using NumPy and manually by formula.

Version: 1.1
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-04-19
"""

from __future__ import annotations

import numpy as np

from utils.input_utils import ask_int_in_range, ask_yes_no
```

```

def create_random_matrix(rows: int, columns: int) -> np.ndarray:
    """Create a random integer matrix.

    Args:
        rows: Number of rows.
        columns: Number of columns.

    Returns:
        NumPy integer matrix.
    """
    # np.random.default_rng() генератор чисел NumPy, получить случайные значения для матрицы.
    rng = np.random.default_rng()

    # rng.integers() создает NumPy-массив случайных целых чисел.
    # low=-50 – минимальное значение включительно. high=51 – верхняя граница не включается,
    # значит максимум будет 50.
    # size=(rows, columns) задает размер матрицы.
    # Возвращается ndarray – основной тип массива в NumPy.
    return rng.integers(low=-50, high=51, size=(rows, columns))

def normalize_by_max_abs(matrix: np.ndarray) -> np.ndarray:
    """Divide all matrix elements by the maximum absolute element.

    Args:
        matrix: Source matrix.

    Returns:
        Normalized matrix.

    Raises:
        ValueError: If maximum absolute value is zero.
    """
    # np.abs(matrix) применяет модуль к каждому элементу матрицы.
    # np.max(...) находит максимальный элемент именно по модулю.
    max_abs_value = np.max(np.abs(matrix))

    # Если вся матрица состоит из нулей, max(abs(A)) будет равен 0. Делить на 0 нельзя,
    # ошибку.
    if max_abs_value == 0:
        raise ValueError("Cannot divide by zero maximum absolute value.")

    # NumPy поддерживает векторизованные операции:
    # matrix / max_abs_value делит сразу каждый элемент матрицы на одно число.
    return matrix / max_abs_value

def calculate_variance_manually(matrix: np.ndarray) -> float:
    """Calculate variance manually by formula.

```

```

Args:
    matrix: Source matrix.

Returns:
    Variance value.
"""
# matrix.ravel() превращает двумерную матрицу в одномерный массив.
# [[1, 2],
#  [3, 4]]
# станет [1, 2, 3, 4].
# удобно считать статистику по всем элементам.
flat_values = matrix.ravel()

# np.mean() считает среднее арифметическое: сумма всех элементов / количество элементов.
mean_value = np.mean(flat_values)

# Для дисперсии нужно найти отклонение каждого элемента от среднего,
# затем возвести эти отклонения в квадрат.
# NumPy делает это сразу для всего массива.
squared_differences = (flat_values - mean_value) ** 2

# np.sum() считает сумму квадратов отклонений.
# flat_values.size — количество элементов.
# Формула дисперсии:
#  $D = \frac{\sum (x_i - \text{mean})^2}{n}$ 
# делим на n, поэтому считаем генеральную дисперсию.
return float(np.sum(squared_differences) / flat_values.size)

def calculate_mode_manually(matrix: np.ndarray) -> float:
    """Calculate mode using NumPy methods."""
    # Мода — это самое часто встречающееся значение.
    # В NumPy нет np.mode. Поэтому используем np.unique(..., return_counts=True).
    # unique_values — массив уникальных значений.
    # counts — сколько раз каждое значение встретилось.
    unique_values, counts = np.unique(matrix, return_counts=True)

    # np.argmax(counts) возвращает индекс самого большого количества повторений.
    mode_index = np.argmax(counts)

    # По этому индексу берем значение, которое встречается чаще всего.
    return float(unique_values[mode_index])

def demonstrate_numpy_initialization() -> None:
    """Demonstrate basic NumPy array initialization methods."""
    print("\nNumPy initialization examples:")

```

```

# np.array() создает NumPy-массив из обычного Python-списка.
# Отличие NumPy array от list:
# list может хранить элементы разных типов: [1, "text", 3.5].
# NumPy array обычно хранит элементы одного типа.
# За счет этого массивы NumPy быстрее и компактнее для вычислений.
example_array = np.array([1, 2, 3, 4])
print(f"np.array([1, 2, 3, 4]): {example_array}")

# np.zeros() создает матрицу заданного размера, заполненную нулями.
zeros_matrix = np.zeros((2, 3))
print(f"np.zeros((2, 3)):\n{zeros_matrix}")

# np.ones() создает матрицу заданного размера, заполненную единицами.
ones_matrix = np.ones((2, 3))
print(f"np.ones((2, 3)):\n{ones_matrix}")

# np.empty() создает массив заданного размера, но не заполняет его конкретными
значениями. Там могут быть старые случайные значения из памяти.
empty_matrix = np.empty((2, 2))
print(f"np.empty((2, 2)):\n{empty_matrix}")

# np.full() создает матрицу заданного размера и заполняет все элементы указанным
значением.
full_matrix = np.full((2, 3), 7)
print(f"np.full((2, 3), 7):\n{full_matrix}")

# np.eye() создает единичную матрицу:
# на главной диагонали стоят 1, все остальные элементы равны 0.
identity_matrix = np.eye(3)
print(f"np.eye(3):\n{identity_matrix}")

# np.arange() создает одномерный массив чисел с шагом.
# Здесь числа от 1 до 10 с шагом 2: 1, 3, 5, 7, 9.
arange_array = np.arange(1, 10, 2)
print(f"np.arange(1, 10, 2): {arange_array}")

def demonstrate_indexing_and_slicing(matrix: np.ndarray) -> None:
    """Demonstrate indexing and slicing.

    Args:
        matrix: Source matrix.
    """
    print("\nIndexing and slicing examples:")

    # matrix[0, 0] – элемент первой строки и первого столбца. В Python индексация начинается
    с 0.
    print(f"First element matrix[0, 0]: {matrix[0, 0]}")

```

```

# matrix[0, :] – первая строка целиком.
# 0 означает первую строку, : означает "все столбцы".
print(f"First row matrix[0, :]: {matrix[0, :]}")

# matrix[:, 0] – первый столбец целиком.
# : означает "все строки", 0 означает первый столбец.
print(f"First column matrix[:, 0]: {matrix[:, 0]}")

# matrix[:2, :2] – срез:
# первые две строки и первые два столбца.
print(f"Slice matrix[:2, :2]:\n{matrix[:2, :2]}")

def run_task_5() -> None:
    """Run task 5."""
    while True:
        print("\nTask 5. NumPy matrix processing.")
        print("Developer: Kruchonok Aleksandra Sergeevna")
        print("Development date: 2026-04-19")

        rows = ask_int_in_range("Enter number of rows from 2 to 10: ", 2, 10)
        columns = ask_int_in_range("Enter number of columns from 2 to 10: ", 2, 10)

        # Создаем исходную случайную целочисленную матрицу A.
        matrix = create_random_matrix(rows, columns)

        # Создаем новую матрицу B по условию варианта:
        # B = A / max(abs(A)).
        normalized_matrix = normalize_by_max_abs(matrix)

        # np.var() считает дисперсию средствами NumPy.
        # По умолчанию np.var делит на n, то есть считает генеральную дисперсию.
        variance_numpy = float(np.var(normalized_matrix))

        # Считаem ту же дисперсию вручную по формуле.
        variance_manual = calculate_variance_manually(normalized_matrix)

        # Мода – самое часто встречающееся значение.
        # Считаem ее через np.unique и np.argmax.
        mode_value = calculate_mode_manually(normalized_matrix)

        print("\nSource integer matrix A:")
        print(matrix)

        # Демонстрация разных способов создания массивов NumPy.
        demonstrate_numpy_initialization()

        # Демонстрация индексации и срезов.
        demonstrate_indexing_and_slicing(matrix)

```

```

print("\nUniversal function examples:")

# np.abs(A) — универсальная функция.
# Она применяется к каждому элементу массива.
print(f"np.abs(A): \n{np.abs(matrix)}")

# np.max(np.abs(A)) — максимальный по модулю элемент.
print(f"np.max(np.abs(A)): {np.max(np.abs(matrix))}")

# Векторизованные операции NumPy:
# A + 10 прибавляет 10 ко всем элементам матрицы.
# A * 2 умножает все элементы матрицы на 2.
print(f"A + 10: \n{matrix + 10}")
print(f"A * 2: \n{matrix * 2}")

print("\nNew matrix B = A / max(abs(A)):")
print(normalized_matrix)

print("\nStatistical functions:")

# np.mean(B) — среднее арифметическое:
# сумма всех элементов / количество элементов.
print(f"np.mean(B): {np.mean(normalized_matrix):.4f}")

# np.median(B) — медиана.
# Медиана — центральное значение отсортированного набора.
# Если количество элементов четное, берется среднее двух центральных.
print(f"np.median(B): {np.median(normalized_matrix):.4f}")

# Мода — самое повторяющееся значение.
# В коде получена через np.unique(..., return_counts=True).
print(f"Mode by np.unique: {mode_value:.4f}")

# np.std(B) — стандартное квадратическое отклонение.
# Это квадратный корень из дисперсии.
print(f"np.std(B): {np.std(normalized_matrix):.4f}")

# np.var(B) — дисперсия:
# среднее значение квадратов отклонений от среднего.
print(f"np.var(B): {variance_numpy:.4f}")

print("\nVariance comparison:")
print(f"Variance by np.var: {round(variance_numpy, 2)}")
print(f"Variance by manual formula: {round(variance_manual, 2)}")

# np.isclose() сравнивает вещественные числа с учетом небольшой погрешности.
# Для float-чисел лучше использовать isclose, а не ==.
if np.isclose(variance_numpy, variance_manual):

```

```

        print("The results match.")
    else:
        print("The results are different because of calculation precision.")

    if not ask_yes_no("\nRepeat task 5? (y/n): "):
        break

```

Работа программы (ввод матрицы 4x5):

```

=====
Choose task number: 5

Task 5. NumPy matrix processing.
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-04-19
Enter number of rows from 2 to 10: 4
Enter number of columns from 2 to 10: 5

Source integer matrix A:
[[-47  31  25 -42 -14]
 [ -8  25 -33  39 -29]
 [-13  22 -10  -7  38]
 [-44 -21  9  -6 -32]]

Indexing and slicing examples:
First element matrix[0, 0]: -47
First row matrix[0, :]: [-47  31  25 -42 -14]
First column matrix[:, 0]: [-47  -8 -13 -44]
Slice matrix[:2, :2]:
[[-47  31]
 [ -8  25]]

New matrix B = A / max(abs(A)):
[[-1.          0.65957447  0.53191489 -0.89361702 -0.29787234]
 [-0.17021277  0.53191489 -0.70212766  0.82978723 -0.61702128]
 [-0.27659574  0.46808511 -0.21276596 -0.14893617  0.80851064]
 [-0.93617021 -0.44680851  0.19148936 -0.12765957 -0.68085106]]

Statistical functions:
np.mean(B): -0.1245
np.median(B): -0.1915
Mode by np.unique: 0.5319
np.std(B): 0.5822
np.var(B): 0.3389

Variance comparison:
Variance by np.var: 0.34
Variance by manual formula: 0.34
The results match.

Repeat task 5? (y/n): 

```

Демонстрация методов NumPy:

```
NumPy initialization examples:
np.array([1, 2, 3, 4]): [1 2 3 4]
np.zeros((2, 3)):
[[0. 0. 0.]
 [0. 0. 0.]]
np.ones((2, 3)):
[[1. 1. 1.]
 [1. 1. 1.]]
np.empty((2, 2)):
[[-7.60543753e+038  3.64172997e+306]
 [ 8.85668651e+106  6.93766018e+182]]
np.full((2, 3), 7):
[[7 7 7]
 [7 7 7]]
np.eye(3):
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
np.arange(1, 10, 2): [1 3 5 7 9]
```

Universal function examples:

```
np.abs(A):
[[47 31 25 42 14]
 [ 8 25 33 39 29]
 [13 22 10  7 38]
 [44 21  9  6 32]]
np.max(np.abs(A)): 47
A + 10:
[[-37  41  35 -32 -4]
 [  2  35 -23 49 -19]
 [-3  32  0  3 48]
 [-34 -11 19  4 -22]]
A * 2:
[[-94  62  50 -84 -28]
 [-16  50 -66 78 -58]
 [-26  44 -20 -14 76]
 [-88 -42 18 -12 -64]]
```


Задание 6. Используя указанный в задании набор данных (datasets) на ресурсе <https://www.kaggle.com/datasets> исследовать основные возможности библиотеки Pandas.

Если какая-то из ссылок не работает, то искать на ресурсе с помощью поиска или в Google (пример запроса: spotify dataset kaggle).

См. Анализ Данных Python методичка.pdf

ЗАДАНИЕ А. Библиотека Pandas. Структуры Series и DataFrame

1. Библиотека Pandas. Импорт библиотеки.
2. Структура Series.
3. Создание Series.
4. Функция display.
5. Доступ к элементам Series с использованием .loc или .iloc.
6. Объект DataFrame. Создание.

№	Датасет	Ссылка на Kaggle	Конкретное задание	Ключевые операции
13	Отзывы IMDB	IMDB Dataset of 50K Movie Reviews	Создайте Series review_length из длин отзывов. Примените лямбда-ф-цию для создания категории длины.	Работа с текстовыми данными, применение функций

ЗАДАНИЕ Б. Основные операции и статистический анализ

Получение информации о датафрейме (по каждому параметру).

Индексация и извлечение данных: статистические методы (во сколько раз среднее значение выбранного показателя с максимальным значением другого параметра (например, самый молодой, самый быстрый и т.д.) выше среднего выбранного показателя с минимальным значением).

№	Датасет	Ссылка на Kaggle	Статистическое задание (формат: определить, во сколько раз ...)
13	Отзывы IMDB	IMDB Dataset of 50K Movie Reviews	Какова средняя длина отзыва (review length) для положительных отзывов (sentiment == 'positive')? Ответ округлите до сотых.

Листинг основного кода программы:

```
""" Laboratory Work No. 4.

Task 6. Pandas Series and DataFrame analysis.

Variant 13:
Analysing of IMDB Dataset of 50K Movie Reviews

Version: 1.5
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-04-21
"""

from __future__ import annotations

from pathlib import Path

import pandas as pd

from utils.input_utils import ask_yes_no

DATA_FILE = Path("data") / "IMDB Dataset.csv"
OUTPUT_DIR = Path("output")
RESULT_FILE = OUTPUT_DIR / "task6_pandas_result.txt"

def load_dataset(filename: Path) -> pd.DataFrame:
    """Load the IMDB dataset from CSV file."""
    if not filename.exists():
        raise FileNotFoundError(
            f"Файл датасета не найден: {filename}. "
            "Нужно скачать IMDB Dataset.csv и положить его в папку data."
        )

    # pd.read_csv считывает CSV-файл и возвращает DataFrame.
    return pd.read_csv(filename)

def categorize_review_length(length: int) -> str:
    """Return review length category."""
    # Категории длины выбраны вручную:
    # short – короткий отзыв,
    # medium – средний отзыв,
    # long – длинный отзыв.
    if length < 500:
        return "short"

    if length < 1500:
```

```

        return "medium"

    return "long"

def analyze_dataset(dataframe: pd.DataFrame) -> tuple[pd.DataFrame, str]:
    """Analyze IMDB reviews dataset and build report."""
    # copy() нужен, чтобы не изменять исходный DataFrame напрямую.
    df = dataframe.copy()

    # Проверяем, что в датасете есть нужные столбцы.
    # review – текст отзыва.
    # sentiment – тональность отзыва: positive или negative.
    required_columns = {"review", "sentiment"}

    if not required_columns.issubset(df.columns):
        raise ValueError("В датасете должны быть столбцы 'review' и 'sentiment'.")

    # Series – это одномерная структура данных Pandas.
    # создается Series review_length: для каждого отзыва считается длина текста в символах.
    review_length = df["review"].str.len()

    # Добавляем Series в DataFrame как новый столбец.
    df["review_length"] = review_length

    # Lambda-функция применяется к каждому значению столбца review_length.
    # В результате создается новый столбец length_category.
    df["length_category"] = df["review_length"].apply(
        lambda length: categorize_review_length(length)
    )

    # value_counts() считает, сколько раз встречается каждое значение.
    sentiment_counts = df["sentiment"].value_counts()
    category_counts = df["length_category"].value_counts()

    # groupby группирует строки по sentiment,
    # затем mean считает среднюю длину отзывов в каждой группе.
    average_length_by_sentiment = df.groupby("sentiment")["review_length"].mean()

    # Разница средних длин между positive и negative.
    # Это нужно для вывода: отличаются ли отзывы по средней длине.
    if {"positive", "negative"}.issubset(average_length_by_sentiment.index):
        positive_mean = average_length_by_sentiment["positive"]
        negative_mean = average_length_by_sentiment["negative"]
        mean_difference = abs(positive_mean - negative_mean)
    else:
        mean_difference = 0

```

```

# .loc обращается к строкам/столбцам по меткам.
loc_example = df.loc[0, ["sentiment", "review_length", "length_category"]]

# .iloc обращается к строкам/столбцам по числовым позициям.
iloc_example = df.iloc[0][["sentiment", "review_length", "length_category"]]

# Чтобы отчет не был огромным, выводим только компактные столбцы.
compact_head = df[["sentiment", "review_length", "length_category"]].head()

report_lines = [
    "Задание 6. Анализ датасета IMDB с помощью Pandas",
    "Разработчик: Kruchonok Aleksandra Sergeevna",
    "Дата разработки: 2026-04-21",
    "=" * 70,
    "",
    f"Размер датасета: {df.shape[0]} строк, {df.shape[1]} столбцов",
    f"Столбцы датасета: {list(df.columns)}",
    "",
    "Первые 5 строк после обработки:",
    str(compact_head),
    "",
    "Пример Series review_length:",
    str(df["review_length"].head()),
    "",
    "Пример .loc для первой строки:",
    str(loc_example),
    "",
    "Пример .iloc для первой строки:",
    str(iloc_example),
    "",
    "Количество отзывов по тональности:",
    str(sentiment_counts),
    "",
    "Количество отзывов по категориям длины:",
    str(category_counts),
    "",
    "Средняя длина отзыва по тональности:",
    str(average_length_by_sentiment),
    "",
    f"Разница средних длин positive и negative: {mean_difference:.2f} символов",
    "",
    "Вывод:",
]

if mean_difference < 100:
    report_lines.append(
        "Средние длины положительных и отрицательных отзывов близки. "
        "Значит, одна только длина текста не является надежным признаком "
        "для определения тональности отзыва."
    )

```

```

    )
else:
    report_lines.append(
        "Средние длины положительных и отрицательных отзывов заметно отличаются. "
        "Это значит, что длина текста может быть дополнительным признаком, "
        "но по одной длине нельзя точно определить тональность."
    )

return df, "\n".join(report_lines)

def save_report(report: str) -> None:
    """Save report to text file."""
    # Создаем папку output, если ее еще нет.
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    # write_text записывает текст в файл.
    # Если файл уже существует, он будет перезаписан.
    RESULT_FILE.write_text(report, encoding="utf-8")

def run_task_6() -> None:
    """Run task 6."""
    while True:
        print("\nЗадание 6. Анализ IMDB Dataset с помощью Pandas.")
        print("Разработчик: Kruchonok Aleksandra Sergeevna")
        print("Дата разработки: 2026-04-21")

        try:
            dataframe = load_dataset(DATA_FILE)
            analyzed_dataframe, report = analyze_dataset(dataframe)

            print("\n" + report)

            save_report(report)

            print(f"\nОтчет сохранен в файл: {RESULT_FILE}")

            print("\nПервые 10 обработанных строк:")
            print(
                analyzed_dataframe[
                    ["sentiment", "review_length", "length_category"]
                ].head(10)
            )

        except FileNotFoundError as error:
            print(error)
            print("\nКак исправить:")
            print("1. Скачай датасет с Kaggle.")

```

```

        print("2. Распакуй архив.")
        print("3. Положи только файл 'IMDB Dataset.csv' в папку data.")

    except Exception as error:
        print(f"Ошибка при обработке датасета: {error}")

    if not ask_yes_no("\nRepeat task 6? (y/n): "):
        break

```

Работа программы:

```

=====
Choose task number: 6

Задание 6. Анализ IMDB Dataset с помощью Pandas.
Разработчик: Kruchonok Aleksandra Sergeevna
Дата разработки: 2026-04-21

Задание 6. Анализ датасета IMDB с помощью Pandas
Разработчик: Kruchonok Aleksandra Sergeevna
Дата разработки: 2026-04-21
=====

Размер датасета: 50000 строк, 4 столбцов
Столбцы датасета: ['review', 'sentiment', 'review_length', 'length_category']

Первые 5 строк после обработки:
   sentiment  review_length  length_category
0  positive         1761             long
1  positive          998             medium
2  positive          926             medium
3  negative          748             medium
4  positive         1317             medium

Пример Series review_length:
0    1761
1     998
2     926
3     748
4    1317
Name: review_length, dtype: int64

Пример .loc для первой строки:
sentiment      positive
review_length      1761
length_category      long
Name: 0, dtype: object

Пример .iloc для первой строки:
sentiment      positive
review_length      1761
length_category      long
Name: 0, dtype: object

```

Количество отзывов по тональности:

```
sentiment
positive    25000
negative    25000
Name: count, dtype: int64
```

Количество отзывов по категориям длины:

```
length_category
medium    31319
long      13719
short      4962
Name: count, dtype: int64
```

Средняя длина отзыва по тональности:

```
sentiment
negative    1294.06436
positive    1324.79768
Name: review_length, dtype: float64
```

Разница средних длин positive и negative: 30.73 символов

Вывод:

Средние длины положительных и отрицательных отзывов близки. Значит, одна только длина текста не является надежным признаком для определения тональности отзыва.

Отчет сохранен в файл: output\task6_pandas_result.txt

Первые 10 обработанных строк:

	sentiment	review_length	length_category
0	positive	1761	long
1	positive	998	medium
2	positive	926	medium
3	negative	748	medium
4	positive	1317	medium
5	positive	656	medium
6	positive	726	medium
7	negative	934	medium
8	negative	681	medium
9	positive	176	short